Working with Data and Visualization

AI Summer Camp – Day 2



Quick Recap: What did we learn so far?

Variables and Datatypes

how to store numbers, text, and True/False values

Inputs and Outputs

get info from the user and show results with input() and print()



Operators

do math and compare values

Conditional Statements

make decisions with if, elif, and else

Quick Recap: What did we learn so far?

Loops and Loop Control

repeat actions with for and while

Functions and Methods

organize code with reusable blocks that can return values



Lists

store an ordered collection of items to access by position

Dictionaries

store key-value pairs for easy lookups and updates

Quick Recap: What did we learn so far?

So far, we've written our Python code in single blocks or cells. But real-world programs, especially complex AI projects, are rarely just one giant file. They are often split into **multiple, smaller Python files** (called **modules**).

Why is this important for large projects?

1. **Organization:** Keeps code tidy and easy to navigate.
2. **Manageability:** Easier to work on smaller, focused pieces of code.
3. **Collaboration:** Multiple programmers can work on different files simultaneously without conflicts.
4. **Avoiding Overwhelm:** Prevents a single file from becoming too long and complex to understand.

Quick Recap: What did we learn so far?

Functions: They help us reuse blocks of code within *one* program.

Remember this function? It takes a number as its input **parameter** and **returns** the calculated square result

But what if we need this function across multiple projects or modules? Do we need to constantly write it over and over again?

```
# Get square of a number (x^2)

def get_square(number):
        square = number * number
        return square
```

Simple Project Folder Structure

- This file contains the main code that starts and controls how your project runs.

main.py

- Files that store extra functions and helper scripts that make your code cleaner and easier to manage.

tools & utilities/

- This folder holds all the files, like text or images, that your project needs to read or work with.

data/

Simple Project Folder Structure

Project/square.py

```
def get_square(number):
    square = number * number
    return square
```

This file acts as a module within our project. It's a dedicated place to store a specific, reusable function: `get_square()`.

Project/main.py

```
from square import get_square
get_square(4)
# output is 4^2 = 16
```

This is our main program file. Instead of redefining `get_square()`, we use the **`import`** statement to bring it in from our *square.py* module.

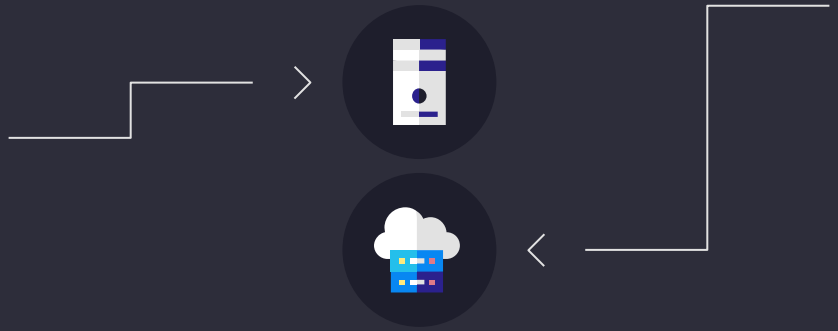
What's an import?

In Python, the import statement is like building a bridge between different parts of your code. It allows you to bring code (functions, variables, classes) from one Python file (called a **module**) into another file where you want to use it.

Imports keep your projects organized, *DRY* (Don't Repeat Yourself), and easy to grow.

Custom Files

reuse your own functions and code saved in other Python files



Libraries

unlock powerful ready-made tools written by others without dealing with their internal implementation

Examples of common libraries



Python Standard Libraries

These are essential collections of code that come **built-in** with Python. You don't need to install them separately; they're ready to import and use right away! They provide common functionalities for everyday programming tasks.

- **math:** For mathematical operations
- **random:** For generating random numbers
- **datetime:** For working with dates and times.
- **os:** For interacting with your computer's operating system (files and folders)



PyPI Third Party Libraries

These are powerful libraries developed by the global Python community. They are not built-in and need to be installed (usually via pip from PyPI, the Python Package Index, like an app store for Python code).

- **NumPy:** For incredibly fast numerical computations and working with arrays
- **Pandas:** For organizing, cleaning, and manipulating tabular data (like spreadsheets).
- **Matplotlib:** For creating professional data visualizations (charts, graphs).

Why do we use libraries?

Libraries contain highly optimized, pre-written code that can replace many lines of manual programming with just one powerful function call. This saves time and reduces errors!

```
scores = [85, 92, 78, ..., 91]
# Imagine a very long list with 1000 scores!

total_sum = 0
count = 0

for score in scores:
    total_sum += score
    count += 1

average = total_sum / count
print(f"Average score is {average}")
# Many lines of code just for average!
```

← **Without libraires**

Vs

with libraries ↓

```
import numpy as np
# Bring in the NumPy toolbox

scores = np.array([85, 92, 78, ..., 91])

average = np.mean(scores)
# One simple line!

print(f"Average score is {average}")
```



Now let's continue on over colab